**RESEARCH STATEMENT**

CMU
PhD program in CS

Amar Shah

---

How can we know that the code we write is correct? How can we trust blackboxes provided by anonymous developers or opaque machine learning algorithms? These questions inform my work at the intersection of Formal Methods, Programming Languages, and Automated Reasoning. I aim to build better verification tools for complicated systems, from rotorcraft aviation to distributed protocols. This is difficult: these problems range from NP-Hard to uncomputable. However, these questions are not only incredibly interesting to me, they are crucial. At CMU, I aim to combine theoretical insights with principled engineering to tame these challenges.

*Stochastic Control Theory:* In Summer 2022, I worked at the University of Maryland Applied Math REU under Professor Maria Cameron, studying stochastic systems. For example, we considered the problem: *is a flying rotocraft stable under environmental perturbations?*

We wanted to study this system through numerical simulation, but ran into the issue that certain perturbations were incredibly rare. To remedy this, we introduced a controller and if this controller was "optimal", we could infer properties of the uncontrolled system.

Earlier work had shown that the optimal controller could be expressed in terms of a different function, the commitor. I spent the early part of the summer numerically approximating the committor using a neural network, but we were still unsure about the generality of this approach. Professor Cameron gave me this challenge: to generalize the optimal controller to other systems. This would require a new analytic proof.

This problem was more difficult than I initially anticipated. Our system was much more general than those considered in prior work. I spent weeks applying standard techniques, but nothing I tried worked. While going through an old control theory paper during one of the last weeks of the REU, I found a clever trick that exploited the symmetry of the system. I was delighted: this was the last step needed to finish the proof. **I was invited to present this result at the Gulf Coast Undergraduate Research Seminar (GCURS) and the Joint Mathematics Meetings (JMM). This culminated in a second-author publication in the journal Communications in Nonlinear Science and Numerical Simulation (CNSNS)**[1] [Yuan et al., 2024].

*SMT solver for Algebraic Datatypes:* When I returned to Berkeley in Fall 2022, I wanted to combine earlier interests in logic with this work ensuring the safety and correctness of complicated real-world systems. I worked under Professor Sanjit Seshia building a Satisfiability Modulo Theories (SMT) solver for Algebraic Data Types (ADTs). ADTs are programming constructs in functional programming languages, but are incredibly useful for modeling a number of automated reasoning problems, including software verification. However, most SMT solvers for ADTs are slow, outdated, and timeout on simple reasoning problems. This is especially disappointing, since more advanced tools rely upon SMT solvers[2].

We tried to come up with a novel, eager approach that would do the bulk of the work upfront to be more efficient. More precisely, we tried to reduce ADTs to a simpler theory Equality & Uninterpreted Functions (EUF). Then we could employ an SMT solver for EUF to solve the reduced formula.

Through experimentation, I realized that the key difficulty in solving these queries was "well-foundedness": no ADT term could refer to itself. This was very difficult to encode in EUF. After much trial and error, we found a finite approximation for this "well-foundedness" which could be encoded into EUF. I presented this in our group meeting and people in the lab were excited by this idea. Now that we had a reduction, we had to build the SMT solver.

In Spring 2023, I submitted this to the Programming Languages Design and Implementation Student Research Competition (PLDI SRC) with my reduction and a simple implementation.

[1] Acceptance Rate: 19%

[2] We added support for ADTs to the verification tool `UCLID5`, using our solver *Algaroba* on the back-end!

With this idea and a proof of concept, I won a Summer Undergraduate Research Fellowship and the EECS Evergreen Research Award to continue this work over the summer. **I was accepted to the PLDI SRC and won first place**.

In Summer 2023, we built a more general solver. By the end, our solver *Algaroba* outperformed the state-of-the-art significantly on all benchmarks, confirming our initial hypothesis. **This culminated in a first author submission accepted at AAAI 2024**[3] [Shah et al., 2024].

*Verification for Distributed Systems:* In Professor Seshia's lab, I worked on a tool for learning properties of distributed systems. We wanted to learn chains of messages that nodes in a distributed system send to one another. We can get positive examples of these message chains by observing the outputs of a system. But to check properties of the system, we need negative examples as well. We use a technique called bootstrapping: we use a coarser method to generate negative examples from positive examples and then a more fine grained method that uses both of these to generate a result. Our tool combined techniques from program synthesis, classical learning theory, and deep learning to learn properties of systems that prior tools could not. It was easily plugged into a larger verification pipeline to prove correctness of these systems. This is an ongoing research direction and we hope to apply this technique to other problems.

*Teaching:* In addition to research, I love teaching. I have spent six terms on course staff for a wide range of classes: Calculus, Discrete Math, Algorithms, and Quantum Computing. Simultaneously, I was a Teaching Assistant for the Berkeley Math Circle Elementary, a program that exposes elementary school students to challenging math. Through all these experiences, I have aimed to inspire the enthusiasm for the material in my students that my teachers once inspired in me. To celebrate this, **I won the Outstanding Graduate Student Instructor award**[4]**.** My research only matters if I work with others and motivate future research. To that end, I want to continue as a teacher and mentor as a PhD student.

*Future Work:* While there has been significant advances in automated reasoning techniques, such as SMT solvers, there are limitations preventing us from applying these tools to verifying large-scale software systems. I aim to work on this by developing techniques that are compositional, and combine learning with reasoning.

Compositionality is an underexploited feature of computer programs in verification. A reasoning tool can break a program down into multiple subparts, verify their correctness, and then combine them to verify the correctness of the entire thing. I have explored these techniques for distributed system verification and am eager to apply this to other domains.

Recent deep learning advances have been applied to a number of domains, but struggle with achieving correctness. Meanwhile, logic-based reasoning problems can guarantee correctness while struggling with scaling. It is natural to combine these two. I have already done this in my work in control theory, where we combined neural networks with analytical results. One exciting direction I have started working in is neurosymbolic approaches to program synthesis: generating programs with a combination of symbolic search and gradient-based optimization.

I am excited about CMU's PhD program because of the very exciting work done by Professors Marijn Heule, Bryan Parno, Jeremey Avigad, and Jan Hoffmann. I am also very interested in Pure and Applied Logic Program. I am very interested in the work Prof. Heule and Avigad do in mathematical theorem proving, especially the recent work on packing numbers (TACAS '23). I believe that problems of this complexity will need to be solved by combining a number of different automated reasoning techniques: SAT/SMT solvers, interactive theorem provers, and perhaps probabilistic methods. I am excited by tools Prof. Parno has developed such as Verus and Vale to verify Rust and low-level code. Prof. Hoffman's work on resource analysis and probabilistic programming as well as their intersection (NomosPro) is a very good fit with my interests.

[3] Acceptance Rate: 23.75%

[4] Awarded to less than 10% of instructors!

# References

[Shah et al., 2024] Shah, A., Mora, F., and Seshia, S. A. (2024). An eager satisfiability modulo theories solver for algebraic datatypes. *Proceedings of the AAAI Conference on Artificial Intelligence*.

[Yuan et al., 2024] Yuan, J., Shah, A., Bentz, C., and Cameron, M. (2024). Optimal control for sampling the transition path process and estimating rates. *Communications in Nonlinear Science and Numerical Simulation (CNSNS)*, 129:107701.